Formalising EML in Lean 4

A post-frontier technical summary
`arXiv:2603.21852`

Bartosz Naskręcki
Adam Mickiewicz University, Poznań / CCAI Warsaw University of Technology
with Aristotle (Harmonic), GPT Pro (OpenAI), and Claude (Anthropic)

12 May 2026

Abstract

This report documents the Lean 4 + Mathlib v4.28 formalisation of *All elementary functions from a single binary operator* by A. Odrzywołek (`arXiv:2603.21852`). The paper introduces the *EML* operator $\mathrm{eml}(x, y) = \exp(x) - \log(y)$ and shows that the two-element basis $\{1, \mathrm{eml}\}$ is a *Sheffer system* sufficient to reconstruct each of 36 primitive functions on a reference calculator. Our artefact seals all 36 paper primitives via literal $\mathsf{EMLTerm}$ (or $\mathsf{EMLTerm}_{\mathbb{C}}$, complex fragment) witness terms whose partial evaluation matches the paper's stated value on a non-empty open subdomain of the natural mathematical domain. The three §G boundary points the paper itself flags ($\sqrt{0}$, $\operatorname{arcosh} 1$, $\operatorname{hypot}(0, 0)$) are additionally sealed by *witness-family* theorems of the shape $\forall \mathrm{env}, [\mathrm{hyp}] \to \exists t \in \mathsf{EMLTerm}, t.\mathrm{eval}_{?}\mathrm{env} = \mathrm{some}\, v$, in which the term t is allowed to depend on the environment. After the post-submission frontier sprint of 2026-05-10/11, the public Lean API exposes 100 theorems: 61 paper-claim theorems (48 EML + 8 EDL + 5 −EML) plus 39 from six frontier modules covering full-domain trig (Path C′), §G boundaries (`GFullFix`), variable-transplant identity depths (`TransplantDepths`), polynomial-class universal minimality (`PolynomialBinary`), and the EDL closed-value closure with a conditional-on-typeclass obstruction scaffold (`EDLClosedVal`). `lake build` finishes sorry-free at 8062 jobs; every public theorem depends only on Mathlib's three standard axioms (`propext`, `Classical.choice`, `Quot.sound`). The artefact is independently re-verified on PCSS Eagle HPC.

Contents

1 Introduction

1.1 The paper

The source paper proves that the binary operator

$$\mathrm{eml}(x, y) \;=\; \exp(x) - \log(y),$$

together with the constant 1, is a Sheffer system for elementary functions: each primitive on a 36-button reference calculator admits a finite *EML reconstruction tree* built from $\{1, \mathbf{x}_n, \mathrm{eml}\}$. The reference calculator hosts six constants (π, e, $\mathbf{i}$, -1, 1, 2), the two variables x and y, the unaries exp, log, inv, $\sqrt{\cdot}$, sin, cos, arctan, sinh, arcosh, . . . , and the binaries $+$, $-$, $\cdot$, $/$, $\log_b$, pow, avg, hypot (36 primitives in total). The paper's central evidence is empirical (a Rust-plus-Mathematica sieve enumerated trees up to fixed complexity and reported per-primitive RPN-length witnesses), supplemented by a paper-level proof sketch (Lemmas 1–3, Corollary 4 of the Supplementary Information).

The paper's Supplementary §2 explicitly notes that "a natural next step would be formalization in Lean 4, but preliminary AI-assisted attempts failed; the extended-value conventions ($\log 0 = -\infty$) and branch-cut reasoning required appear to exceed current automation capabilities." The artefact described here demonstrates that, with the right architectural choices, formalisation *is* feasible end-to-end.

1.2 Why the paper's framing makes this hard

Three Lean-specific constraints shape every architectural decision.

- **Totality.** Lean kernel functions must be total. Mathlib defines `Real.log 0 = 0` (the "junk value") and similarly $\sqrt{x} = 0$ for $x \leq 0$. Short Supplementary witnesses such as $0 = L(1)$ and $-1 = L(1) - 1$ that implicitly use $\log 0 = -\infty$ *fail* at face value: the natural EML composition evaluates them to 1, not 0.
- **Branch cuts.** Every reconstruction beyond the elementary $e = \mathrm{eml}(1, 1)$ uses log on values that cross or sit near the principal branch. "Valid on its principal branch" is one sentence in the paper proof sketch; in Lean it is a hypothesis whose 11-field side-condition bundle must be discharged at every mkLog application.
- **Term-level vs. function-level.** A real-valued identity $\log z = \mathrm{eml}(1, \mathrm{eml}(\mathrm{eml}(1, z), 1))$ is one theorem; lifting it to an inductive grammar $\mathsf{EMLTerm}$ so that the universality claim is $\exists t \in \mathsf{EMLTerm}, \forall \mathrm{env}, t.\mathrm{eval} \cdots = \log(\mathrm{env}\, 0)$ is another. We need both, and Mathlib has no built-in scaffolding for either.

1.3 Headline result

After two phases of work (a chunked-decomposition phase 1 in April 2026, archived in `process_archive/REPORT_2026_04_27.pdf`; a structural-compiler plus Path C′ phase 2 in May 2026; a frontier sprint phase 3 on 2026-05-10/11), the artefact today seals:

- All 36 paper primitives via literal $\mathsf{EMLTerm}$ or $\mathsf{EMLTerm}_{\mathbb{C}}$ witnesses on a non-empty open subdomain of the natural domain;
- The three §G boundary points via witness-family theorems (in `GFullFix.lean`) where the term depends on the environment;
- Full natural domain for sin, arctan, tan via Path C′ range-reduction by substitution;
- 8/36 Sheffer-cousin primitives for the paper's EDL cousin and 5/36 for −EML (`Sheffer.lean`, `EDLClosedVal.lean`);
- Variable-transplant identity terms at depths $4k$ for every $k \in \mathbb{N}$ plus negative closures at depths 1, 2 (`TransplantDepths.lean`);
- A polynomial-class first cut of paper §5 universal minimality (`PolynomialBinary.lean`): no bivariate polynomial binary operation can equal `Real.exp` on all of $\mathbb{R}$.

The public Lean API now exposes 100 **machine-checked theorems** (61 original paper claims + 39 frontier). `lake build` finishes sorry-free at 8062 jobs. Every public theorem depends only on the three Mathlib-standard axioms; the `K_count` theorems are pure `rfl`-decidable and depend on no axioms at all (see Section 6).

2 Architecture

2.1 The three-grammar pipeline

The compositional core is a three-language pipeline. Each box is an inductive Lean type; each arrow is a total function returning `Option`-valued partial evaluations.

Layer	What it holds
F36	36-primitive source language with partial denotation
EL	Intermediate language: $\{\exp, \log, \text{neg}, \text{inv}, +, -, \cdot, /, \dots\}$, real partial eval
EMLTerm	Single-operator grammar: $T ::= 1 \mid \mathbf{x}_n \mid \text{eml}(T, T)$, real partial eval
$\mathsf{EMLTerm}_{\mathbb{C}}$	Complex-coefficient extension, same syntax, $\mathbb{C}$ semantics

The two compilers F36 $\xrightarrow{\texttt{translate?}}$ EL and EL $\xrightarrow{\texttt{compile}}$ EMLTerm are defined in `Framework/Compilers/F36ToEL.lean` and `Framework/Compilers/ELToEML.lean`; each preserves partial evaluation. The complex extension lifts every real-fragment witness to $\mathsf{EMLTerm}_{\mathbb{C}}$ via a syntactic identity homomorphism $\iota\colon \mathsf{EMLTerm} \to \mathsf{EMLTerm}_{\mathbb{C}}$ and provides Euler-style witnesses for the trig family (§3.3).

2.2 Partial evaluation as the totality work-around

Every EMLTerm evaluation returns `Option` $\mathbb{R}$, with $\text{eml}(a, b)$ returning `none` whenever $b \le 0$. This is the key design move that side-steps the $\log 0 = 0$ junk-value problem: nested `eml(a, b)` *never* evaluates to a junk value at a boundary; it returns `none` instead. Concretely:

```
noncomputable def EMLTerm.eval? (env : Nat -> Real)
    : EMLTerm -> Option Real
  | .one      => some 1
  | .var n    => some (env n)
  | .eml a b => (eval? env a).bind fun va =>
                  (eval? env b).bind fun vb =>
                    if vb <= 0 then none
                    else some (Real.exp va - Real.log vb)
```

Every `paper_claim_<f>` theorem states an existential of the shape

$$\exists\, t \in \mathsf{EMLTerm}.\ \forall \text{env} \in (\mathbb{N} \to \mathbb{R}),\ [\text{precondition on env}],$$
$$t.\text{eval}_?\,\text{env} = \text{some}\,(f(\text{env}\,0,\ \text{env}\,1,\ \dots)).$$

The precondition is empty when f is a full-natural-domain primitive; otherwise it restricts to the open subdomain of validity (e.g. $x > 0$ for $\sqrt{\cdot}$).

2.3 Path C′: the witness-family shape

Two architectural problems force a generalisation of the existential shape:

1. **Trig widening.** The Euler-bridge witnesses for sin, arctan, tan work only on symmetric subdomains around 0 because the intermediate $\log_{\mathbb{C}}$ argument crosses the principal branch outside that subdomain.
2. **§G boundary points.** A single environment-independent EMLTerm witness for $\sqrt{0}$, $\text{arcosh}\,1$, or $\text{hypot}(0, 0)$ fails because the natural composition evaluates to 1 at the boundary, not 0.

The fix in both cases is the same: **flip the quantifiers**. Move from the original

$$\exists t \in \mathsf{EMLTerm}.\quad \forall \text{env},\ t.\text{eval}_?\,\text{env} = \text{some}\,(f(\dots))$$

to a *witness-family* statement

$$\forall \text{env}.\quad [\text{hyp}] \to \exists t \in \mathsf{EMLTerm},\ t.\text{eval}_?\,\text{env} = \text{some}\,(f(\dots)),$$

where the choice of t is allowed to depend on env. For trig, the dependence is a period number $k \in \mathbb{Z}$ such that $x - k\pi$ lands in the local-witness domain. For §G, the dependence is a case split: the constant-zero term mkZero on the boundary, the existing narrow witness off the boundary.

The witness-family statement is mathematically equivalent to what the paper's compiler does in prose: it "manually corrects the i-sign" per region of x at code-generation time, which is exactly choosing a different witness per environment.

2.4 The structural compiler vs. per-primitive witnesses

A second design decision is whether to seal each primitive *individually* (one hand-tuned witness per `paper_claim_<f>`) or to seal a *structural compiler* that produces witnesses uniformly from a higher-level grammar.

The artefact does both, but the bulk uses the structural compiler. The compiler F36 $\to$ EL $\to$ EMLTerm takes any F36-expression and mechanically produces a EMLTerm witness; the per-primitive theorems are then thin existential shells that instantiate the compiler with the corresponding F36-expression. Hand-tuned witnesses are kept for the closed numeric constants (π, i, 0, 2, $-i$) and the trig family, where domain-specific tricks (the Cayley quotient for tan, the substitution route $\arcsin(x/\sqrt{1+x^2})$ for arctan) compress the witness drastically.

The cost of uniformity is K-count blow-up: $K(\log_b) = 9\,929\,087$ in the compiler output versus a few hundred nodes for a hand-tuned witness. We treat that gap as informative rather than worrying: the paper's K-counts are an *upper bound on necessary size*, and our machine-checked figures are the size of a *specific mechanically-produced* witness.

2.5 Real-to-complex term lift

Many complex witnesses (the trig family, most §G fixes) need to use a real value $-x$ inside a complex composition. Constructing $-x$ from scratch in $\mathsf{EMLTerm}_{\mathbb{C}}$ is expensive; instead, we use a syntactic homomorphism `EMLTerm.toComplex` that lifts any real-fragment EMLTerm tree to $\mathsf{EMLTerm}_{\mathbb{C}}$ unchanged, preserves the size measure, and satisfies $\mathrm{eval}_? = (\mathrm{eval}_? \cdot \mathrm{ofReal})$. Combined with the public closed-constants $\mathrm{realize}_{\mathbb{C}}^{\{0,2,-i\}}$ this lets every trig witness reuse the already-sealed real fragment for arithmetic sub-expressions and avoids redoing branch-cut work for each new primitive.

3 What is sealed

3.1 Coverage scoreboard

Family (count)	Single structural witness	Witness-family (full domain)
Atoms (7)	full natural domain	—
i (1)	full (via $\mathrm{realize}_{\mathbb{C}}^{\mathbf{i}}$)	—
Real unary (7)	full natural domain	—
$\sqrt{\cdot}$ (1)	$(0, \infty)$	$\forall x \geq 0$
Hyperbolic (5)	full natural domain	—
arcosh (1)	$(1, \infty)$	$\forall x \geq 1$
Binary (7)	full natural domain	—
hypot (1)	$\mathbb{R}^2 \setminus \{(0,0)\}$	$\forall (x, y)$
cos, arccos, arcsin (3)	full natural domain	—
sin, arctan, tan (3)	narrow positive / negative subdomains	full natural domain (Path C′)

Net. 36/36 paper primitives sealed. Three §G boundary points sealed by witness families. Three trig primitives widened to full natural domain by Path C′. The headline existential statement holds for every primitive.

3.2 The §G witness-family seal

The module `Framework/GFullFix.lean` provides three theorems:

```
theorem paper_claim_sqrt_full :
    forall env : Nat -> Real, 0 <= env 0 ->
      exists t : EMLTerm, t.eval? env = some (Real.sqrt (env 0))

theorem paper_claim_arcosh_full :
    forall env : Nat -> Real, 1 <= env 0 ->
      exists t : EMLTerm, t.eval? env = some (Real.arcosh (env 0))

theorem paper_claim_hypot_full :
    forall env : Nat -> Real,
      exists t : EMLTerm,
        t.eval? env = some (Real.sqrt (env 0 ^ 2 + env 1 ^ 2))
```

The proof of each is a case split on whether the environment is at the boundary. On the boundary the witness is the constant-zero term mkZero; off the boundary the witness is the corresponding narrow paper-claim term obtained from the existential it produces.

The module `Framework/StructuralLimitsEReal.lean` confirms the mathematical correctness of the boundary values in extended-real arithmetic using ENNReal.log (which sends 0 to $\perp$ rather than the junk value 0). The two modules together close the gap: `GFullFix` is the Lean-level witness-family seal, and `StructuralLimitsEReal` is the independent confirmation that the value those witnesses produce is the mathematically correct one.

3.3 Path C′: full-real-domain trig

The three Path C′ widenings deliver

- `paper_claim_sin_full` via $\sin x = \cos(\pi/2 - x)$;
- `paper_claim_arctan_full` via $\arctan x = \arcsin(x/\sqrt{1+x^2})$;
- `paper_claim_tan_full` via the $\tan x = \tan(x - k\pi)$ period reduction.

The shared infrastructure is the substitution operator $\mathrm{subst}_0 : \mathsf{EMLTerm}_{\mathbb{C}} \to \mathsf{EMLTerm}_{\mathbb{C}} \to \mathsf{EMLTerm}_{\mathbb{C}}$ that replaces every $\mathbf{x}_0$ in a host term with a given substitute term, together with the foundational lemma $\mathrm{ADDsafe}_{\mathbb{C}}^{\mathrm{ofReal,\,ofReal}}$ which discharges the 11-field $\mathrm{ADDsafe}_{\mathbb{C}}$ precondition for $\mathrm{mkAdd}_{\mathbb{C}}$ whenever both arguments are real-valued. This lets period shifts via repeated $\mathrm{mkAdd}_{\mathbb{C}}$ of fixed real period constants stay entirely in the real fragment, where the $\arg = \pi$ branch-cut trap never appears.

3.4 Closed numeric constants and the imaginary unit

Five reusable $\texttt{EMLRealization}_{\mathbb{C}}$ packages ($\mathrm{realize}_{\mathbb{C}}^{\{0,2,-\mathbf{i},\mathbf{i},\pi\}}$) live in the `Closures/Constants` module of the complex framework. Each consists of a literal $\mathsf{EMLTerm}_{\mathbb{C}}$ tree plus a partial-evaluation proof that the tree returns the constant when given any environment. These are the building blocks for the trig witnesses, the Path C′ period shifts, and the §G `mkZero` term.

For the hand-tuned closed constants our K-counts match the paper's hand-tuned Table 4 figures to the unit: $K(0) = 7$, $K(2) = 19$, $K(-\mathbf{i}) = 127$, $K(\mathbf{i}) = 407$, $K(\pi) = 233$.

3.5 Sheffer cousins: EDL and $-$EML

The paper presents EML, EDL, and $-$EML as a "family" of Sheffer operators (paper §3.1, equation block `label{Sheffers}`) but proves completeness only for EML; the cousins are confirmed empirically via the Mathematica/Rust `VerifyBaseSet` procedure.

The artefact's `Framework/Sheffer.lean` scaffolds the two paper-named cousins as inductive grammars with partial-evaluation semantics:

- `EDLTerm` with the binary $\mathrm{edl}(x, y) = \exp(x)/\log(y)$ paired with the constant e;
- `NegEMLTerm` with the binary $-\mathrm{eml}(y, x) = \log(x) - \exp(y)$ paired with the constant $-\infty$, which requires a parallel `NegEMLTermE` grammar over EReal to seal the $-\infty$ constant itself.

Eight of the 36 EDL primitives and five of the 36 $-$EML primitives are sealed in this module (Plans D and E in `OPEN_QUESTIONS.md`). The non-trivial EDL witnesses include Aristotle's three-step composition for $\log x$ and the $\mathrm{edl}(\mathrm{D8}(x), \mathrm{D4}(y))$ construction for division.

3.6 The conditional ceiling scaffold

`Framework/EDLClosedVal.lean` formalises a closure result for the closed-EDL value set:

```
inductive EDLClosedVal : Real -> Prop where
  | one      : EDLClosedVal 1
  | e_const  : EDLClosedVal (Real.exp 1)
  | edl      : {a b : Real} ->
               EDLClosedVal a -> EDLClosedVal b ->
               Real.log b /= 0 ->
               EDLClosedVal (Real.exp a / Real.log b)

theorem edl_closed_eval_in_closedVal :
    forall {t : EDLTerm}, t.IsClosed ->
    forall (env : Nat -> Real) {v : Real},
      t.eval? env = some v -> EDLClosedVal v
```

The closure theorem is proved unconditionally by induction on the EDLTerm structure. To turn it into per-primitive non-membership statements (no closed EDL term evaluates to -1, 2, or $\frac{1}{2}$), we need three transcendence-style facts whose proof is currently out of Mathlib's reach. We package them as a Lean typeclass:

```
class EDLTranscendenceBarrier : Prop where
  neg_one_not_closed : ~ EDLClosedVal (-1)
  two_not_closed     : ~ EDLClosedVal 2
  half_not_closed    : ~ EDLClosedVal ((1 : Real) / 2)
```

Three obstruction corollaries (e.g. `no_closed_edl_neg_one`) take the typeclass as a hypothesis. We *intentionally provide no instance*, so the corollaries are scaffolded but not closed. A future Mathlib Schanuel-style transcendence result would discharge the typeclass and close them; until then, any user invoking these corollaries is signing for the conjecture explicitly. This is the "conditional ceiling scaffold" framing used throughout the artefact's public documentation.

4 The frontier sprint

Four research-grade directions were identified by an independent GPT Pro architectural review (separate context, no shared scratchpad) in early May 2026. A focused two-day sprint on 2026-05-10/11 produced substantive Lean content for all four, adding 39 public theorems.

4.1 Direction 1: variable-transplant identity depths (SI §1.5 #5)

Module: `Framework/TransplantDepths.lean` (9 theorems).

The paper's identity term has depth four; the Supplementary Information asks (open question #5) whether other depths admit identity terms. `TransplantDepths.lean` proves:

- **Affirmative family.** For every $k \in \mathbb{N}$, an explicit $\mathsf{EMLTerm}$ term constructor of depth $4k$ that evaluates to $\mathrm{env}\,0$ on every real environment.
- **Negative closures for $d = 1$ and $d = 2$.** No $\mathsf{EMLTerm}$ of either depth is identity on every environment.
- **$d = 3$ statement.** The proposition `NoIdentityAtDepthThree_conjecture`, recording the next open case. Aristotle proved it in a simplified single-atom grammar (saved at `chunks/090_no_identity_depth_3/result.lean` in the archive); the canonical-grammar port (our two-atom $\{1, \mathrm{var}\}$ grammar) is the natural follow-up.

4.2 Direction 2: §G boundary points

Covered in §3.2.

4.3 Direction 3: Plan D conditional ceiling

Covered in §3.6.

4.4 Direction 4: paper §5 universal minimality, polynomial-class first cut

Module: `Framework/PolynomialBinary.lean` (2 theorems).

The paper §5 universal-minimality question asks whether *any* single-binary-operator scaffold $\{c, B\}$ can match EML's coverage. The general question is open; the polynomial-class first cut is fully proved.

Definition 4.1 (Polynomial binary)**.** A binary operation $B\colon \mathbb{R} \to \mathbb{R} \to \mathbb{R}$ is *polynomial* when there exists a bivariate polynomial witness $P \in \mathrm{MvPolynomial}(\mathrm{Fin}\,2, \mathbb{R})$ such that $B(x, y) = P(x, y)$ for all $x, y \in \mathbb{R}$.

Lemma 4.2 (Composition lemma)**.** *If B is polynomial and t is any free term over B with constants and a single variable, then there exists a univariate polynomial $Q \in \mathbb{R}[X]$ such that $t.\mathrm{eval}\,B\,(\mathrm{const}\,x) = Q(x)$ for every $x \in \mathbb{R}$.*

Theorem 4.3 (Polynomial-binary impossibility)**.** *No polynomial binary operation, freely composed with constants and a single variable, can equal* Real.exp *on every input.*

Proof sketch. By the composition lemma, the candidate witness reduces to a univariate polynomial Q with $Q(x) = \exp(x)$ on every x. Then $Q(x)/\exp(x) = 1$ identically. But the Mathlib lemma `Polynomial.tendsto_div_exp_atTop` says this ratio tends to 0 at infinity, contradicting the constant-1 identification. □

This is a single class of rule-outs. Rational, semialgebraic, Pfaffian, and real-analytic classes each need their own impossibility argument, which remain paper-open.

4.5 Direct-macro alternative witnesses: an honest finding

Module: `Framework/AlternativeWitnesses.lean` (9 alternatives + 9 K-counts).

For the seven non-trivial binary primitives we built a parallel "direct-macro" set of witnesses bypassing the structural compiler. The expectation was K-count reduction; the actual finding is that the direct-macro witnesses have *identical* K-counts to the compile-output ones, because the structural compiler already uses these macros internally. The high K-counts reflect the genuine cost of full-natural-domain coverage, not a compiler inefficiency.

The module was named `CompactWitnesses.lean` during development and renamed to `AlternativeWitnesses.lean` after the finding to avoid misleading future readers.

5 Limitations and architectural findings

5.1 Plan B: custom complex-log branch is architecturally infeasible

The paper at line 333 sketches an alternative route to full real-domain trig coverage by "manually correcting the i-sign" inside the compiler. The natural Lean translation is to introduce a custom branch-of-log primitive in $\mathsf{EMLTerm}_{\mathbb{C}}$ and route certain trig witnesses through it.

This route foundered on a structural fact: $\mathsf{EMLTerm}_{\mathbb{C}}$'s partial evaluation ($\mathtt{EMLTerm}_{\mathbb{C}}\mathtt{.eval?}$ in `Framework/Complex/Term.lean`) hard-codes Mathlib's principal `Complex.log`. There is no place in the grammar definition to substitute a different branch convention. Adding one would require either (a) extending the grammar with a parameterised $\mathrm{mkLog}_{\mathbb{C},\theta}$ constructor (and propagating the parameter through every macro), or (b) reinterpreting the existing constructor through a custom evaluator (which breaks every existing `eval?` lemma).

We concluded the route was not worth the architectural cost, and adopted Path C′ (range-reduction by substitution) instead. Path C′ recovers the same full-real-domain coverage with no grammar change, at the cost of moving from single-witness statements to witness-family statements. The detailed finding is recorded in `OPEN_QUESTIONS.md` §B.0.

5.2 Conjectural items intentionally left open

- **EDL transcendence barrier.** The three obstruction corollaries for -1, 2, $\frac{1}{2}$ are conditional on `EDLTranscendenceBarrier` (§3.6). No instance is provided.
- **Universal minimality beyond polynomial.** Theorem 4.3 rules out polynomial binaries only. The rational, semialgebraic, Pfaffian, and real-analytic classes each remain open.
- **Variable-transplant depth 3.** The canonical-grammar non-existence at $d = 3$ is recorded as a `Prop` (`NoIdentityAtDepthThree_conjecture`). Aristotle has the proof in a simplified single-atom grammar.
- **SI §1.5 questions 1, 2, 3, 4, 6, 7.** The paper's own open-questions list. Out of artefact scope.
- **§4.3 gradient-based symbolic regression.** Out of artefact scope (Mathlib has no gradient-flow / projection / floating-point $\leftrightarrow$ symbolic infrastructure).

6 Verification evidence

6.1 Build

`lake build EML` finishes successfully at 8062 jobs, with warnings confined to two cosmetic linters (`linter.unusedSimpArgs`, `linter.unusedVariables`). A verbatim build-tail transcript is in `lambda_lab/proofs/eml/2603_21852/VERIFICATION_EVIDENCE.md`.

6.2 Axiom audit

The audit script `lean_workspace/EML/AxiomCheck.lean` runs `#print axioms` on every headline theorem. The verbatim output (also in `VERIFICATION_EVIDENCE.md`) shows that every public theorem depends only on Mathlib's three standard axioms: `propext`, `Classical.choice`,

`Quot.sound`. The `K_count_*` theorems go further: they are pure `rfl`-decidable and depend on *no* axioms at all.
What this rules out.

- No `sorry` hides inside the chain (the axiom set is the closed Mathlib-standard one).
- No project-side `axiom` declaration was introduced.
- The `EDLTranscendenceBarrier` typeclass, the one place where the artefact *consciously* uses an unproved hypothesis, does not appear in the axiom set of the unconditional closure theorem `edl_closed_eval_in_closedVal`. The three conditional corollaries take it as a parameter and would print it in their axiom list if asked; by design no instance is provided.

6.3 Repository hygiene

`grep -rn 'sorry|admit' lean_workspace/EML/ -include=*.lean` returns nothing matching an actual tactic site. The four false-positive matches are all comment text describing the word "sorry" (e.g. "a permanent `sorry`" from the phase-1 report). The CI hook `scripts/check_no_sorries.sh` enforces this on every commit.

6.4 Independent re-verification

PCSS Eagle HPC re-verifies the artefact on a different machine and a different Lean toolchain installation. Most recent run: job 7 041 555 on 2026-05-07, 88 files, 0 failures, 42 s wall time. The submission scripts are in `eagle_scripts/`.

7 Reproducing the verification

7.1 Local

```
git clone https://github.com/nasqret/eml-formalization
cd eml-formalization/lambda_lab/proofs/eml/2603_21852/lean_workspace
lake build EML                          # ~30-60 min cold cache
lake env lean EML/AxiomCheck.lean       # axiom audit
```

The first build pulls ~ 6 GB of Mathlib oleans and finishes in 30–60 minutes; subsequent builds are incremental.

7.2 On PCSS Eagle (HPC)

```
# From a node where ~/pl0414-02/scratch/elan_dl/
# has the Lean toolchain tarball
cd /mnt/storage_5/scratch/pl0414-02
sbatch verify_all.sbatch     # parallel verify across 24 cores
```

The SLURM scripts are in `eagle_scripts/` and handle (a) Lake olean cache rebuild and (b) parallel re-verify.

8 Where the formalisation diverges from the paper

Three architectural moves separate our artefact from a literal transcription of the paper's prose.

1. **Partial evaluation as junk-value avoidance.** The paper treats $\log 0 = -\infty$ as routine; we make every nested eml-evaluation return `none` outside the natural domain of log. The bridge theorems are stated as "if F36-eval returns `some v`, then there exists an EMLTerm term that also returns `some v`"; we never claim equality at a boundary point.

2. **Three §G boundary points as witness families.** The paper's witness for $\sqrt{0}$, arcosh 1, and hypot(0, 0) would implicitly use the $\log 0 = -\infty$ convention; Mathlib's $\log 0 = 0$ blocks this. We use the quantifier flip described in §2.3. The paper itself (line 342 of `EML.tex`) explicitly notes this Lean-specific issue.
3. **Path C′ instead of the paper's "manual i-sign correction".** The paper's prose at line 333 sketches a custom branch of $\log_{\mathbb{C}}$. We found this architecturally infeasible in Lean (§5) and used range-reduction by substitution instead. Both approaches deliver full-real-domain trig coverage; the paper's is implicit in compiler code that chooses different witness shapes per region of x, ours is explicit in the quantifier flip.

9 Conclusion and acknowledgements

The Lean 4 + Mathlib v4.28 artefact described in this report seals all 36 primitives of the paper's reference calculator via literal EMLTerm or $\mathsf{EMLTerm}_{\mathbb{C}}$ witness terms on a non-empty open subdomain of their natural mathematical domain. The three §G boundary points the paper itself flags are additionally sealed by witness-family theorems in which the witness depends on the environment. After the post-submission frontier sprint, the public API exposes 100 machine-checked theorems covering the original paper claims, full-domain trig, §G boundary fixes, variable-transplant identity depths, and a polynomial-class first cut of paper §5 universal minimality. The build is sorry-free at 8062 jobs and every public theorem depends only on Mathlib's three standard axioms.

The architectural shifts that made this possible are recorded in detail in `AUTHOR_SUMMARY.md` and `notes/proof_structure.pdf`; the live status of every open item is in `OPEN_QUESTIONS.md`; the fresh re-verification evidence is in `VERIFICATION_EVIDENCE.md`.

Acknowledgements

- **Andrzej Odrzywołek** (Jagiellonian University) — the source paper. Thanks for both the discovery of the EML operator and for the careful description of the §G boundary issue (paper line 342) which spared us a great deal of confusion when we first hit it in Lean.
- **Aristotle** (Harmonic AI) — proof search for many individual chunks during the phase-1 chunked decomposition and the frontier sprint. The full chunk archive is in `process_archive/chunks/`.
- **GPT Pro** (OpenAI) — three independent-context architectural reviews. Recommended the structural-compiler architecture, the Cayley-quotient route for tan, the public closed-constants packaging, Path C′, the witness-family quantifier flip, the four frontier-sprint directions, and the pre-announcement punch list. Transcripts in `process_archive/gpt_pro_bundle/`.
- **Claude Code** (Anthropic) — orchestration, scaffolding, integration of Aristotle returns, post-submission trig widenings, and frontier-sprint module assembly.
- **Codex** (OpenAI) — paraphrase and informalisation of mathematical statements during paper-to-grammar translation.
- **Mathematica** — enumeration and witness candidate search during phase-1 decomposition.
- The **Mathlib community** — the underlying Lean library.

Phase-1 process history. An earlier 60-page report dated 27 April 2026, describing the chunked decomposition approach (since superseded by the structural compiler and the frontier sprint), is preserved at `process_archive/REPORT_2026_04_27.pdf` for the audit trail.